```
41
42      @Benchmark
43      @Fork(value = 1)
44      @Threads(1)
45      @Warmup(iterations = 1, time = 10, timeUnit = TimeUnit.SECONDS)
46      @Measurement(iterations = 10, time = 10, timeUnit = TimeUnit.SECONDS)
47      @BenchmarkMode(Mode.AverageTime)
48      @OutputTimeUnit(TimeUnit.NANOSECONDS)
49      public void vectorComputation(ScalarAndVectorMulComparision.StateObj stateObj) {
50          int i = 0;
51          int upperBound = SPECIES.loopBound(stateObj.a.length);
52          for (; i < upperBound; i += SPECIES.length()) {
53              // FloatVector va, vb, vc;
54              // read from array in batches 0-7, 8-15 and so on
55              FloatVector va = FloatVector.fromArray(SPECIES, stateObj.a, i);
56              FloatVector vb = FloatVector.fromArray(SPECIES, stateObj.b, i);
57              // vectorized multiplications and negation
58              FloatVector vc = va.mul(va)
59                      .add(vb.mul(vb))
60                      .neg();
61              vc.intoArray(stateObj.c, i);
62          }
63          for (; i < stateObj.a.length; i++) {
64              stateObj.c[i] = (stateObj.a[i] * stateObj.a[i] + stateObj.b[i] * stateObj.b[i]) * -1.0f;
65          }
66      }
67  }
```

FIgure 6: Vector computation

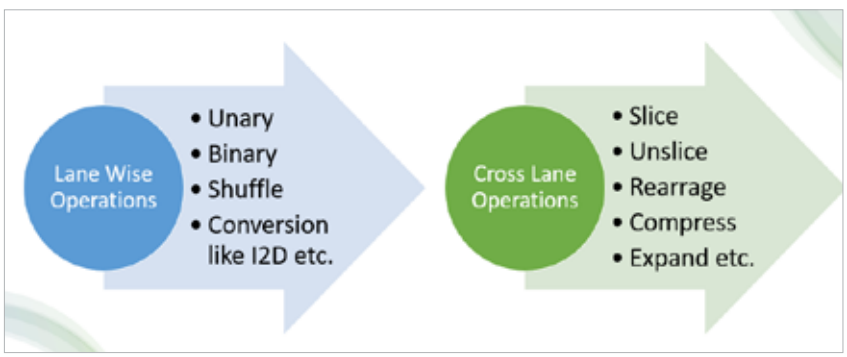


Figure 7: Vector operators: Lane-wise and cross-lane

meaning 2 iterations can be performed in a vectorized manner handling 16 floats at a time, in a total of 32 floats. The remaining 4 floats, which do not fit into the vectorized flow, should be handled in a scalar or sequential manner as seen from line number 63. This section of the code, which handles the remaining data elements that don't qualify for vectorized computation, is called the 'tail part'.

The process involves reading from the array into vectors one at a time — from indices 0 to 15, then 16 to 31, and so on. The calculations, including multiplications and negation, are then performed in a vectorized, data-parallel manner, which is more efficient than executing them sequentially. The results are stored back into the resultant array or vector, denoted as 'vc'.

## Vector operations: Lane-wise vs cross-lane

The Java Vector API supports multiple operations and functions, with the concept of a 'lane'. These operations can be categorised based on the resultant lane length as lane-wise and cross-lane operations.

When an operation produces a result with the same length as the input vectors, it is referred to as a lane-wise operation. It yields results with the same lane count as the source vectors. Lane-wise operations include unary, binary, shuffle, and conversions like I2D, etc.

In addition to lane-wise operations, there are also cross-lane operations. These may result in vectors with a lesser lane count than the source vectors, depending on how the operation consolidates or processes the data across lanes. Slice, unslice, rearrange, compress, and expand, etc, are cross-lane operations.